Polymorphic Type Inference

What is Parametric Polymorphism?

Parametric polymorphism means a function works for **all types**, not just one. In SML, the function `map` has type `('a -> 'b) -> 'a list -> 'b list`. This says: "Give me *any* function from type A to type B, and a list of A's, and I will return a list of B's." The compiler does not need to know what A or B actually are—the same code works for all types.

The problem: "for all types" is sometimes too general. Different languages solve this differently:

- **C++ templates**: The compiler only type-checks on the specific types you actually use. If you call `add<int>`, it checks that `int` supports `+`. It is not truly "for all T"—it is "for all T that you actually instantiate." Each type gets its own specialized copy of the code.
- **Java generics**: You can write `<T extends Comparable<T>>` to restrict T to types that implement `Comparable`. The compiler type-checks the body once against the bound, and you can only instantiate with subtypes of that bound.
- **SML**: Type variables written `'a` mean "any type." The special notation `''a` means "any type where equality (`=`) is defined." For more complex constraints (like ordering), SML uses **functors**: you provide a module (called a "structure") that implements a "signature" (interface), and the functor produces a specialized module.

How Type Inference Works (Algorithm)

The goal of type inference is to figure out the types of all expressions *without* the programmer writing any type annotations. Here is the algorithm step by step:

1. **Assign fresh type variables**: Every parameter and every return type that is not explicitly annotated gets a fresh, unknown type variable (like `'a`, `'b`, etc.).
2. **Walk the AST**: Recurse down the abstract syntax tree just like normal type checking would.
3. **Unify instead of checking**: On the way back up, instead of checking "are these two types equal?" (which would fail because we have unknowns), we **unify** the types we have with the types we expect. Unification either discovers new information about type variables, confirms types already match, or fails (meaning the program is ill-typed).
4. **Collect substitutions**: Each successful unification may produce a mapping (e.g., "`'a` must be `int`"). These mappings are accumulated and applied to all subsequent unifications.

The Unification Algorithm

Unification takes two types and tries to make them equal by finding substitutions for type variables.

Base cases (simple situations):

- **Type variable with a concrete type**: If we are unifying `'a` with `int`, we learn that `'a` must be `int`. Return the mapping $\{\texttt{'a} \mapsto \texttt{int}\}$.
- **Same type with itself**: Unifying `int` with `int` succeeds with no new information. Return $\emptyset$.
- **Type constructor mismatch**: If the two types have different structures—for example, trying to unify an arrow type (`int -> bool`) with a tuple type (`int * bool`)—this is a **type error** ("Tycon mismatch").
- **Unary type constructors like `list`**: To unify `'a list` with `int list`, strip off the `list` wrapper and unify the arguments: unify `'a` with `int`.

Recursive case (for compound types like `T1 -> T2` and `S1 -> S2`):

```
unify(T1 -> T2, S1 -> S2) =
  let m1 = unify(T1, S1)        (* unify left parts *)
      T2' = apply(m1, T2)       (* MUST apply m1 *)
      S2' = apply(m1, S2)       (* before continuing *)
      m2 = unify(T2', S2')      (* unify right parts *)
  in  combine(m1, m2)
```

CRITICAL RULE: Before unifying the right-hand parts (T2 and S2), you **must apply the substitution** learned from unifying the left-hand parts. If you skip this step, you can get inconsistent results—the left and right halves might give conflicting assignments to the same type variable.

Concrete examples to build intuition:

- `unify(int * 'a, int * string)`: Left parts are both `int`, so $m_1 = \emptyset$. Right parts: unify `'a` with `string` gives $\{\texttt{'a} \mapsto \texttt{string}\}$. Final answer: $\{\texttt{'a} \mapsto \texttt{string}\}$.
- `unify(int -> 'a, 'b * string)`: Arrow vs. tuple is a type constructor mismatch. **FAIL**.
- `unify('a * 'a, int * string)`: Left gives $\{\texttt{'a} \mapsto \texttt{int}\}$. Apply to right: unify `int` with `string`. Mismatch → **FAIL**. (This correctly rejects: you cannot use the same type variable for both `int` and `string`.)
- `unify('a list, int list)`: Strip `list`, unify `'a` with `int`. Result: $\{\texttt{'a} \mapsto \texttt{int}\}$.

Worked Example: Type Inference for the len function

`fun len list = case list of [] => 0 | y::z => 1 + len z`

Step 1—Assign fresh type variables: `list` gets type `'a`, the return type gets `'b`. In the second pattern, `y` gets `'c` and `z` gets `'d`. So `len : 'a -> 'b`.

Step 2—Unify from the bottom of the AST upward:

The recursive call `len(z)`: `z` has type `'d` and `len` expects `'a`. Unify `'d` with `'a` → learn `'a = 'd`.

The expression `1 + len z`: The plus operator requires `int * int -> int`. The result of `len z` is `'b`. Unify `'b` with `int` → learn `'b = int`. So `len` returns `int`.

The pattern `y::z`: The cons operator has type `'e * 'e list -> 'e list`. Unify `'e * 'e list` with `'c * 'd` → learn `'e = 'c` and `'d = 'c list`.

The pattern `[]`: Has type `'f list`. Unify with `'c list` → `'f = 'c`. Both cases of the case expression must return the same type: they both return `int`. Consistent.

The input `list` had type `'a = 'd = 'c list`. Return type `'b = int`.

Final result: `len : 'c list -> int` (i.e., `'a list -> int`). The function takes a list of anything and returns an integer. This makes sense—`len` counts elements regardless of their type.

Practice Final Q1—Type Inference

`fun f(w,x) = case x of [] => [] | y::z => w(y)::f(w,z)`
Step 1—Assign type variables: `w:'a`, `x:'b`, return:`'c`, `y:'d`, `z:'e`.
Step 2—Unification (work from bottom up):

- Match `x` with `[]` (nil has type `'f list`): learn `'b = 'f list`.
- Match `y::z` with the cons constructor (`'h * 'h list -> 'h list`): learn `'d = 'h` and `'e = 'h list`.
- Match `x` with the cons result (`'h list`): so `'f = 'h`.
- `w` is applied to `y`, so `w` must be a function: `'a = 'h -> 'i`.
- The recursive call `f(w,z)` returns something, and we cons `w(y)` onto it. From the cons, `'i = 'j` and the return type `'c = 'j list`.
- Both case branches must have the same type: first branch is `'k list`, second is `'j list`, so `'k = 'j`.
- Check that the recursive call's argument types are consistent: `('h -> 'j) * 'h list = ('h -> 'j) * 'h list`. Yes.

Final type: $f : (\texttt{'h} \to \texttt{'j}) \times \texttt{'h list} \to \texttt{'j list}$
This is exactly the `map` function! It takes a function and a list, applies the function to each element, and returns the resulting list.
Part 2: `fun f(x) = f` fails the **occurs check** (explained below).

The Occurs Check

Whenever unification is about to create a mapping from a type variable `'b` to some type T, it must first check that `'b` does *not* appear anywhere inside T. If it does, the type would be infinitely recursive, which is not allowed.

Example: Consider `fun f x = f`. We assign `f : 'a -> 'b`. The body of `f` is just `f` itself, which has type `'a -> 'b`. So we need to unify the return type `'b` with `'a -> 'b`. But `'b` appears inside `'a -> 'b`—this would mean `'b = 'a -> ('a -> ('a -> ...))`, an infinite type. The occurs check catches this and reports a **type error**.

Hungry Types (A Workaround for the Occurs Check)

SML datatypes allow recursive types through constructors, and these do *not* trigger the occurs check:

```
datatype 'a hungry = F of ('a -> 'a hungry)
```

Now `fun f x = F(f)` type-checks: `f : 'a -> 'a hungry`. The constructor `F` expects `'a -> 'a hungry` and returns `'a hungry`. Everything is finite and well-typed because the recursion goes through the datatype constructor `F`.

Why "hungry": You can pattern-match `F` to extract the inner function, apply it to a value to get another `'a hungry`, and keep "feeding" it values forever. Useful for: print sinks (keep accepting values to print), random number generators (keep producing values), and the Y combinator.

Options Gotcha

A common type error: `case ... of ... => NONE | ... => 42`. The first branch has type `'a option` and the second has type `int`. Since `option` and `int` are different type constructors, unification fails (tycon mismatch).

Fix: Write `SOME 42` instead of `42`, so both branches have type `int option`.

Subtyping

Subtyping determines when one type can be safely used in place of another. The key rules depend on whether you **read from** or **write to** a data structure:

- **Covariant** (same direction as subtyping): Applies when you can only **read** from the structure. If `Lion <: Cat` (Lion is a subtype of Cat), then `Lion list <: Cat list`. Reading a Lion from a list where you expect a Cat is safe—every Lion is a Cat.
- **Contravariant** (opposite direction): Applies when you **write to** the structure. Function arguments are contravariant: `Cat -> int <: Lion -> int`. A function that can handle any Cat can certainly handle a Lion (since Lions are Cats), but a function that only handles Lions cannot handle arbitrary Cats.
- **Invariant** (no subtyping allowed): Applies when you both read *and* write. `Lion ref` is **NOT** a subtype of `Cat ref`. If it were, you could write a Frog (also a Cat) into the ref, then read it expecting a Lion—a type error at runtime. The combination of reading and writing forces exact type matching.

Function types in detail: A function `A -> B` is **contravariant in A** (the argument, which gets "written into" the function) and **covariant in B** (the return value, which gets "read out" of the function).

Read-only references: If a reference were read-only (`const ref`), then covariance would apply—`Lion const_ref <: Cat const_ref`.

MIPS Registers and Calling Convention

MIPS has 32 registers. Here is the full table and what each is used for:

Register	Name	Purpose and Notes
r0	zero	Always contains 0 (hardwired by hardware)
r1	AT	Assembler temporary; used by pseudo-instructions (e.g., BLT decomposes to SLT + branch using r1)
r2–3	v0–v1	Function return values
r4–7	a0–a3	First 4 function arguments (additional arguments go on the stack)
r8–15	t0–t7	**Caller-saved** temporaries
r16–23	s0–s7	**Callee-saved** (function must save/restore if it uses them)
r24–25	t8–t9	More caller-saved temporaries
r26–27	k0–k1	Reserved for the OS kernel
r28	gp	Global pointer (global variables accessed as GP + offset)
r29	sp	Stack pointer
r30	fp	Frame pointer
r31	ra	Return address (set by `jal` instruction)

Usable registers: Approximately 25, after excluding r0 (always zero), r1 (assembler), r26–28 (kernel/global).

Caller-saved vs. callee-saved analogy: Think of gym treadmills. T registers are like ones *you* wipe down before using (the *caller* saves its own values before making a call, because the callee may overwrite them). S registers are like ones the *previous user* wipes after finishing (the *callee* saves their original values and restores them before returning).

Function Prologue (executed before the function body)

The prologue sets up the stack frame so the function has space for local variables and saved registers:

1. `SP -= frame_size`: Allocate space on the stack by decrementing the stack pointer.
2. Save the old frame pointer (FP) to the stack so we can restore it later.
3. `FP = SP + frame_size`: Set the new frame pointer to point to the top of this frame.
4. Save any callee-saved (S) registers that this function will modify. Only save the ones we actually use—no need to save all eight if we only clobber s0 and s1.
5. Save the return address (RA)—but **only if this function calls other functions** (non-leaf). A leaf function (one that makes no calls) will not have RA overwritten, so saving it is unnecessary.

Function Epilogue (executed at the end of the function)

The epilogue undoes the prologue and returns control to the caller:

1. Restore any callee-saved (S) registers we saved earlier.
2. Restore RA from the stack (if we saved it).
3. Reset the stack pointer: either `SP = FP` or `SP += frame_size`.
4. Restore the old frame pointer from the stack.
5. Execute `jr $ra` to jump back to the caller.

What the Caller Does

Before a call: Save any caller-saved (T) registers whose values you still need after the call returns. Move the first four arguments into a0–a3 (any additional arguments go on the stack). Execute `jal target` to jump to the function and save the return address in RA.

After the call returns: Restore the T registers you saved. The function's return value is in v0.

Why the Frame Pointer Matters

1. **Fixed reference point**: The frame pointer stays constant throughout a function's execution. The stack pointer, on the other hand, can change when you use `alloca()`, declare variable-length arrays, or push arguments for nested calls. Variables accessed as `FP - offset` have stable addresses; `SP + offset` would change as SP moves.
2. **Variable-size stack frames**: C allows `int arr[n]` where `n` is determined at runtime. The stack pointer moves unpredictably, but the frame pointer remains fixed.
3. **Debugging**: Debuggers walk the call stack by following the chain of saved frame pointers (a linked list of frames).

Optimization: If a function has no dynamic stack allocation (no `alloca`, no VLAs), the frame pointer can be omitted. This saves a few instructions in the prologue/epilogue and frees up a register.

Static Links and Escaping Variables

The Problem: Nested Functions Accessing Outer Variables

In Tiger (and other languages with nested functions), an inner function can read and modify variables from an enclosing function. These variables cannot simply live in registers, because there is no guarantee that the register will still hold the right value when the inner function executes (especially with recursion). So these variables must live in **stack frames**.

The naive idea of "just follow the chain of old frame pointers" (the dynamic call chain) does not work. The reason is that recursion creates unpredictable numbers of frames. The old FP chain follows the **dynamic call order** (who called whom at runtime), not the **lexical nesting** (which function is textually inside which in the source code). These are different things.

How Static Links Solve This

Each stack frame stores a **static link (SL)**—a pointer to the stack frame of the function that *textually encloses* it in the source code (its "static parent").

Implementation in Tiger: The static link is passed as an implicit first argument (in register a0). Inside the function, it is stored in the frame like any other escaping variable.

To access a variable N levels up the nesting: Follow exactly N static links. This always takes exactly N steps, regardless of how deep the recursion is. That is the key advantage over the dynamic chain.

Rules for What Static Link to Pass

- **Calling a child function** (a function nested directly inside you): Pass your **FP**, because you are its static parent.
- **Calling a sibling function** (at the same nesting level as you): Pass your **static link**, which is `MEM(FP)`. You share the same parent.
- **Calling yourself recursively**: Pass your **static link** (`MEM(FP)`), since you and your recursive copy have the same parent.
- **Calling a function N levels up**: Follow N static links starting from your frame.

Practice Final Q2—Static Links

Nesting structure: Level 0: top-level `let`. Level 1: `odd` and `f`. Level 2: `g` and `h` (both nested inside `f`).
The static link is stored at the frame pointer location in each frame.

Call	SL passed	Reason
f calls g	**FP**	f is g's parent (calling a child)
g calls h	**MEM(FP)**	g and h are siblings (pass f's FP, which is g's SL)
g calls g	**MEM(FP)**	recursive call (same parent f)
h calls f	**MEM(MEM(FP))**	2 levels up: h → f → top-level
g calls odd	**MEM(MEM(FP))**	2 levels up: g → f → top-level

Deep nesting example: If `foo` is inside `h`, which is inside `g`, which is inside `f`, and `foo` needs to access variable `a` from `f`'s frame: follow 3 static links (foo → h → g → f). This is always exactly 3 hops regardless of how many recursive calls are on the stack.

Escaping Variables

A variable "escapes" if it cannot be kept in a register and must live in the stack frame:

- **In Tiger**: A variable escapes if it is used by a nested function. The nested function needs a stable memory address to find it via static links.
- **In C**: A variable escapes if its address is taken with `&x`. A register does not have a memory address.

Escape analysis: Walk the AST and record the nesting depth where each variable is declared. If a variable is referenced at a deeper nesting depth than where it was declared, mark it as escaping. Non-escaping variables can potentially be allocated to registers, which is much faster than accessing memory.

Intermediate Representation (IR)

Why Use an IR?

1. **N+M instead of N×M**: If you have N source languages and M target architectures, building a direct compiler for each pair requires $N \times M$ compilers. With an IR, you only need N front-ends (language → IR) and M back-ends (IR → machine code), for a total of $N + M$. This is much more scalable.
2. **Incremental step**: Translating directly from AST to assembly is a huge conceptual leap. The IR sits halfway between them, making both translations simpler.

Property	AST	IR	Assembly
Structure	Tree	Tree	Linear sequence
Functions	Can be nested	Flat list of fragments	Top-level only
Variables	Named identifiers	Infinite supply of temps	∼25 physical registers
Scope	Nested/hierarchical	Flat	Flat
Types	int, string, etc.	Everything is a 32-bit word	byte, half-word, word
Control flow	if/while/for	Two-target CJUMP	One-target branches

IR Datatypes

The IR has two categories: **statements** (which perform side effects but produce no value) and **expressions** (which produce a value).

Statements (side effects, no value):

- **SEQ(s1, s2)**: Execute statement s1, then execute statement s2. Sequencing.
- **LABEL(l)**: Declares a label at this position in the code. Labels are targets for jumps.
- **JUMP(e, label_list)**: Unconditional jump. The expression `e` computes the target address, and `label_list` lists all possible targets (for the benefit of later analysis). Usually just one label: `JUMP(NAME(l), [l])`. An **empty list** signals the end of a function (`jr $ra`). Multiple labels are used for jump tables.
- **CJUMP(relop, e1, e2, true_label, false_label)**: Conditional jump with **two targets**. Evaluates the comparison (`relop` is one of $<, \leq, >, \geq, =, \neq$) and jumps to the appropriate label. Unlike assembly, there is no implicit fall-through—both targets are explicit.
- **MOVE(dst, src)**: Copy the value of `src` into `dst`. The destination must be either a TEMP (register write) or a MEM (memory store). This dual use is a design flaw—it should have been two separate constructs.
- **EXP(e)**: Evaluate the expression `e` purely for its side effects and throw away the result.

Expressions (produce a value):

- **BINOP(op, e1, e2)**: Binary arithmetic or logic operation (add, subtract, multiply, etc.).
- **MEM(e)**: Memory access at address `e`. When MEM appears as the *destination* of a MOVE, it is a **store**. Everywhere else, it is a **load**.
- **TEMP(t)**: A register from an infinite pool. Later, register allocation maps these to physical registers.
- **ESEQ(s, e)**: Execute statement `s` for its side effects, then evaluate expression `e` for its value. This is how expressions can have side effects in the IR.
- **NAME(l)**: The address of a label. Used to reference functions and data.
- **CONST(n)**: An integer constant.
- **CALL(f, args)**: A function call. `f` is typically `NAME(label)` and `args` is a list of expressions.

Fragments

A **fragment** is a top-level unit of assembly output. There are two kinds:

- **String fragment**: A label paired with string data (e.g., for string literals).
- **Function fragment**: A function body (IR tree) paired with frame information.

Translation of the whole program produces a list of fragments.

Ex / Nx / Cx Wrappers

When translating AST nodes to IR, each translation result is wrapped in one of three constructors:

- **Ex** (expression): Wraps an IR expression. Used for things evaluated for their value, like `a + b`.
- **Nx** (no-result): Wraps an IR statement. Used for things done purely for side effects, like `a := 3`.
- **Cx** (conditional): Wraps a *function* of type `(true_label, false_label) -> statement`. Used for things that control flow, like `x < 4`. When you give it two labels, it produces a CJUMP that goes to one or the other.

The rule is: return whichever wrapper is *most natural* for the expression. `a + b` → Ex, `a := 3` → Nx, `x < 4` → Cx.

Conversions Between Ex, Nx, and Cx

Sometimes you need to use a result in a different context than its natural form. Here are the conversions:

unEx(Ex(e)): Simply return `e`—already an expression.

unEx(Nx(s)): `ESEQ(s, CONST 0)`—do the statement, then evaluate to 0. (The 0 is a meaningless placeholder.)

unEx(Cx(f)): This is the interesting one. We need to turn a conditional (true/false branching) into a 0/1 integer value:

```
r = newTemp(); t1 = newLabel(); f1 = newLabel()
Ex(ESEQ(seq[
  MOVE(TEMP r, CONST 1),   (* optimistically set result to 1 (true) *)
  f(t1, f1),               (* the CJUMP: go to t1 if true, f1 if false *)
  LABEL f1,                (* false branch: *)
  MOVE(TEMP r, CONST 0),   (* override result to 0 *)
  LABEL t1                 (* true branch: result stays 1 *)
], TEMP r))                (* final value is whatever r holds *)
```

unCx(Ex(e)): `CJUMP(e ≠ 0, true_label, false_label)`. Treat any nonzero value as true.

unNx(Ex(e)): `EXP(e)`—evaluate the expression and discard its value.

Helper function: `fun seq [] = EXP(CONST 0) | seq [s] = s | seq (s::rest) = SEQ(s, seq rest)`. This builds a chain of SEQ nodes from a list of statements.

Practice Final Q3—IR Translation

Variable locations: $x \to$ InReg t_1, $y \to$ InReg t_2, $z \to$ InFrame -4 (meaning z is at FP $-$ 4 in memory), $a \to$ InReg t_3 (pointer to array base).

Translating `y := 1 + x`:

`MOVE(TEMP` t_2`, BINOP(PLUS, CONST 1, TEMP` t_1`))`

This says: compute $1 + x$ and store the result in t_2 (which holds y).

Translating `a[y] := f(z)`:

```
MOVE(
MEM(BINOP(PLUS, TEMP t3, BINOP(TIMES, TEMP t2, CONST 4))),
CALL(LABEL(f), [MEM(BINOP(PLUS, FP, CONST −4))]))
```

The left side computes the memory address: array base (t_3) plus index (t_2) times word size (4 bytes). The right side calls function `f` with argument z, which lives in memory at FP -4 (so we load it with MEM).

Translating a record field access `r.z` (where z is the 3rd field):

`MEM(BINOP(PLUS, TEMP r, CONST 8))`

Fields are laid out consecutively, 4 bytes each, 0-indexed. The 3rd field (index 2) is at offset $2 \times 4 = 8$.

Translating Control Flow

If-Then-Else

Translation takes three inputs: the condition (use `unCx` to get a CJUMP-generating function), the then-clause (use `unEx` to get its value), and the else-clause (use `unEx` to get its value).

```
c' = unCx(cond); t' = unEx(then_clause); e' = unEx(else_clause)
r = newTemp; lt = newLabel; lf = newLabel; le = newLabel
Ex(ESEQ(seq[
  c'(lt, lf),            (* CJUMP: go to lt if true, lf if false *)
  LABEL lt,              (* true branch starts here *)
  MOVE(TEMP r, t'),      (* store then-value into result register *)
  JUMP(NAME le, [le]),   (* skip over false branch *)
  LABEL lf,              (* false branch starts here *)
  MOVE(TEMP r, e'),      (* store else-value into result register *)
  LABEL le               (* both branches converge here *)
], TEMP r))              (* final value is whatever r holds *)
```

There are 7 statements in the sequence. For a statement-only if (no value needed), use `unNx` instead—the moves store 0, which is harmless.

Expression sequences with side effects: Tiger's `(x := x+1; x+3)` translates to an ESEQ: the statement part performs the assignment, and the expression part evaluates `x+3` for the final value.

While Loops

Naive translation (two branches per iteration—bad):

```
L_start: if NOT cond, goto L_end (* conditional branch *)
        body
        goto L_start            (* unconditional branch *)
L_end:
```

For 1 million iterations: 1,000,001 conditional branches + 1,000,000 unconditional branches = 2,000,001 total branches.

Optimized translation (one branch per iteration—good):

```
  JUMP(NAME L_test, [L_test])   (* one-time jump to test *)
L_body: body
L_test: CJUMP(cond, L_body, L_done) (* only branch per iteration *)
L_done:
```

1 initial jump + 1,000,001 conditional branches = 1,000,002 total branches. This saves approximately 1 million branches by placing the test at the end of the loop so the body falls through to the test naturally.

Break statements: Translate to `JUMP(NAME(done_label), [done_label])`. The compiler tracks the current loop's done label using a `label option`: `NONE` if we are outside any loop, `SOME(label)` if inside one.

For Loops—The Max-Int Edge Case

Tiger for-loops have **inclusive** upper bounds (the loop variable takes every value from `low` to `high`, including `high`). If `high = maxint` (the largest representable integer), a naive `i := i + 1` after the last iteration causes integer **overflow**, wrapping around to a negative number and creating an infinite loop.

Correct compilation that avoids this:

```
i := low; h := high
if i > h goto L_done      (* handle zero-iteration case *)
L_body: body
  if i < h goto L_incr    (* STRICTLY less-than check *)
  goto L_done              (* if i == h, this was the last iteration *)
L_incr: i := i + 1
  goto L_body
L_done:
```

The key insight: check `i < h` (strictly less than) *before* incrementing. If `i == h`, we have just completed the last iteration and must *not* increment (that would overflow).

Arrays

Memory layout: An array is stored as `[size | elem0 | elem1 | ...]`. The pointer returned to the program points to `elem0` (not to the size field). The size is at `MEM(array_ptr - 4)`.

Creating an array: Call `malloc((size + 1) * 4)` to get space for the size field plus all elements. Store the size at offset 0. Increment the pointer by 4 (so it points past the size field). Fill in the initial values.

Indexing: `a[i]` translates to `MEM(base + i * 4)`. Important: evaluate the index expression into a temporary first! If the index has side effects (like `a[x := x+1; x]`), evaluating it twice would cause the side effect to happen twice.

Bounds checking: Check $0 \leq$ index $<$ MEM(array_ptr $-$ 4). This requires two CJUMPs. If either check fails, jump to an error label that aborts the program.

Optimization note: Ideally, the IR would have an explicit "bounds check" primitive so that later optimization passes could recognize and eliminate redundant checks. For example, inside `for i := 0 to n`, the check $i \geq 0$ is always true. But Appel's IR uses plain CJUMPs for bounds checks, making them indistinguishable from other conditionals. Later passes cannot identify and remove them.

Records

To create a record: call `malloc`, then MOVE each field's value to MEM at the appropriate offset. To access a field: `MEM(base + offset)`, where the offset is known at compile time (field positions are fixed by the type declaration). No bounds checking is needed because the compiler knows exactly which fields exist.

Switch/Case Compilation Strategies

- **If-else chain**: Test each case value in sequence. O(n) comparisons. Fine for small switches (say, 3–5 cases).
- **Binary search**: Sort the case values and do a binary search. O(log n) comparisons.
- **Jump table**: Create an array of code addresses (labels), indexed by the case value. A single array lookup jumps directly to the right case. O(1). Works great when case values are **dense** (e.g., 0, 1, 2, ..., n) but wastes huge amounts of space for sparse values (e.g., 0, 9000, 57000).
- **Hybrid**: Use binary search to narrow down to a dense range, then use a jump table within that range.

Why JUMP has a label list: For jump tables, the target is computed dynamically (`MEM(i * 4 + tableBase)`), and the label list enumerates all possible targets. This lets later analysis phases (like CFG construction) know where control can flow to.

Pattern Matching Compilation

SML datatypes: each constructor is assigned an integer tag (e.g., NONE = 0, SOME = 1). Data is stored as a struct with an `int tag` field followed by a union of payloads. Pattern matching compiles to: check the tag (using if-else on integers), extract the payload, and recurse for nested patterns.

Critical Rule: Never Reuse IR Sub-trees

Never duplicate an IR sub-tree. If a sub-tree contains a label, duplicating it means two copies of the same label in the assembly—an error. If a sub-tree contains side effects, duplicating it means the side effects happen twice. Always evaluate an expression *once*, store the result in a TEMP, and use the TEMP multiple times.

Purifying and Linearizing the IR

Removing ESEQ (Purifying Expressions)

The goal of this phase is to eliminate ESEQ nodes by "bubbling" all side effects up to the top level. After purification, every statement is followed by pure expressions with no embedded side effects.

Rule 1—Flatten nested ESEQs:

`ESEQ(s1, ESEQ(s2, e))` becomes `ESEQ(SEQ(s1, s2), e)`.

In English: do s1 then s2 (combined into one statement block), then evaluate e.

Rule 2—Push ESEQs out of BINOPs:

Consider `BINOP(+, ESEQ(s, e1), e2)`. The ESEQ is buried inside a BINOP, and we need to lift it out.

- **If s and e2 are independent** (s does not change any value that e2 reads): Simply lift: → `ESEQ(s, BINOP(+, e1, e2))`. Safe because s does not affect e2.
- **If they might interfere** (or we cannot prove independence): We must evaluate e1 *before* s happens, by saving it in a temporary: → `ESEQ(SEQ(MOVE(TEMP t, e1), s), BINOP(+, TEMP t, e2))`. The safe version always works but adds register pressure because `t` must remain live across `s`.

Alias Analysis (Determining Whether Two Expressions Are Independent)

If no memory is involved: Check which registers the expression reads versus which registers the statement writes. If the sets are disjoint, they are independent.

If memory is involved: We need to determine "do two pointers point to the same location?" This is **undecidable** in general.

Possible answers from an alias analysis: "Yes, definitely the same", "No, definitely different", or "**May alias**" (sometimes same, sometimes different). We must use **false positives** (report "may alias" when in reality they do not alias) to stay safe. A false negative (saying "no alias" when they actually do) would allow incorrect reordering.

Practical approaches:

1. **Conservative**: Assume all memory accesses may interfere. Safe but pessimistic—misses optimization opportunities.
2. **Type-based**: Different types cannot alias. Used by GCC's `-fstrict-aliasing` flag. An `int*` and a `float*` point to different things.
3. **Allocation-site**: Pointers from different `malloc` calls point to different objects. Requires data flow analysis to track pointer origins.

For the Tiger compiler: the "commute" function returns true/false for independence. Returning false always ("everything may interfere") is pessimistic but correct.

Hoisting Function Calls

`f(g(x), h(y))` must be translated carefully. If we evaluate `g(x)` and then `h(y)` as nested calls, the second call will overwrite the argument registers (a0–a3) and return value register (v0) that the first call set up.

Solution: Hoist (pull out) nested calls to the top level:

```
t1 = g(x)    (* evaluate first, store result in temp *)
t2 = h(y)    (* evaluate second, store result in temp *)
result = f(t1, t2) (* now use the saved results *)
```

This isolates each call so they do not interfere with each other's registers. Evaluation order must be preserved (left to right in Tiger) for correctness when calls have side effects.

Linearizing (Choosing Block Order)

After purification, the IR is split into **basic blocks**. We need to arrange these blocks into a linear sequence in memory. The key requirement is that the **false branch of every CJUMP must fall through** to the next block (because that is how MIPS conditional branches work—they only specify the "taken" target).

BFS (breadth-first order) is bad: Blocks are laid out level by level, but execution typically follows one deep path. BFS creates many long jumps and short straight-line sequences.

DFS (depth-first order) is good: Follows execution paths deeply, creating longer straight-line sequences. Fewer extra jumps needed.

Topological sort does not work: The control flow graph has cycles (from loops), so there is no topological order.

Branch flipping: If DFS happens to place the "true" successor immediately after the CJUMP instead of the "false" successor, we can fix this by negating the comparison operator (e.g., $<$ becomes $\geq$, $=$ becomes $\neq$) and swapping the branch labels.

Profile-guided optimization: Branch directions are rarely 50/50. Loop back-edges are almost always taken. DFS should follow the *likely* path to keep hot code contiguous. GCC provides `__builtin_expect(expr, val)`; C++20 adds `[[likely]]` and `[[unlikely]]` attributes.

I-cache benefits: Placing hot-path blocks close together improves instruction cache locality. Rarely-executed blocks (error handlers, exception paths) should be placed far away to avoid polluting the cache.

Instruction Selection

The Maximal Munch Algorithm

Instruction selection translates IR trees into assembly instructions. The algorithm works top-down:

1. At the current IR node, find the **largest** IR subtree pattern that matches a single assembly instruction. "Largest" means consuming the most IR nodes in one instruction.
2. **Recurse** on any remaining children (parts of the IR not consumed by the matched pattern) to get their values into registers.
3. Each recursive call returns the register where its result lives.
4. **Emit** the matched instruction using the registers from the recursive calls.

There are two mutually recursive functions: **munch_stm** handles statements (no register returned) and **munch_exp** handles expressions (returns the register containing the result).

SML's pattern matching is ideal for implementing this: each pattern clause corresponds to one instruction template. You start with simple patterns (one IR node = one basic instruction) and progressively add larger patterns for optimization.

Key instruction patterns (MIPS):

- `MEM(BINOP(PLUS, e, CONST c))` matches a single `LW rd, c(rs)`. The constant offset is folded into the load instruction, saving a separate add.
- `BINOP(PLUS, e, CONST c)` matches `ADDI rd, rs, c`. The constant is encoded as an immediate operand.
- `MOVE(MEM(BINOP(PLUS, e1, CONST c)), e2)` matches `SW rs, c(rd)`.
- `MEM(e)` alone matches `LW rd, 0(rs)` (load with zero offset).
- `CONST(c)` matches `LI rd, c` (load immediate).

Worked Example: Instruction Selection for a[i]

The IR tree for this assignment is:
`MOVE(MEM(BINOP(+, a, BINOP(*, i, CONST 4))),`
`BINOP(*, BINOP(+, MEM(BINOP(+, z, CONST 8)), CONST 27), CONST 3))`
Applying maximal munch top-down, then emitting instructions bottom-up:
`LI t100, 4` (MIPS has no multiply-immediate, so load the constant 4 first)
`MUL t101, i, t100` (compute $i \times 4$, the byte offset for array index i)
`ADD t102, a, t101` (compute the target address: array base + offset)
`LW t103, 8(z)` (this matches `MEM(BINOP(+, z, CONST 8))` as a single load-with-offset! We get z's 3rd field in one instruction)
`ADDI t104, t103, 27` (this matches `BINOP(+, e, CONST 27)` as add-immediate!)
`LI t105, 3` (load the constant 3)
`MUL t106, t104, t105` (compute the final value: (z.field3 + 27) * 3)
`SW t106, 0(t102)` (store the result at the array location)
Optimizations that fired: The load-with-offset pattern (`LW t103, 8(z)`) saved one ADD instruction compared to loading z, loading 8, and adding them. The add-immediate pattern (`ADDI`) saved one LI instruction. Without these combined patterns, we would need 2 additional instructions.

Multiplication Strength Reduction

MIPS has no multiply-immediate instruction, so multiplying by a constant always requires loading the constant into a register first. But multiplication is also slow (3–4 cycles versus 1 cycle for ALU/shift operations). We can replace multiplications with cheaper shift/add combinations:

- **Multiply by a power of 2** (e.g., $\times 4$): Use left shift (`SLL rd, rs, 2`). One instruction, 1 cycle.
- **Multiply by 3**: Shift left by 1 (multiply by 2), then add the original value. Two instructions.
- **Multiply by 127**: Shift left by 7 (multiply by 128), then subtract the original value. Two instructions.
- **Many set bits in the constant**: Just use MUL—the shift/add decomposition would require too many instructions.

Instruction Selection Output Data Types

The output of instruction selection is a list of objects of three types:

- **OPER**: A general instruction. Contains an assembly template string with placeholders (`'s0`, `'d0` for source/destination registers), lists of source and destination temps, and a jump field. If `jump = NONE`, the instruction falls through. If `jump = SOME([])`, it is the end of the function (`JR RA`). If `jump = SOME(labels)`, those are the possible branch targets.
- **MOVE**: A register-to-register copy. Kept as a separate type because if the register allocator later assigns the source and destination to the *same* physical register, the move can be **deleted entirely**. This optimization is not possible for other instructions (you cannot delete a multiply even if source and destination happen to match).
- **LABEL**: A label marker (no actual instruction emitted, just a position).

JAL (function call) clobbers: A `JAL` instruction must list **all caller-saved registers** (t0–t9, a0–a3, v0, ra) as destinations, because the called function may overwrite any of them. This ensures the register allocator knows those values are lost across the call.

Register Pressure and Instruction Ordering

In the worked example above, the left-side address (t102) must stay live while all the right-side computation happens. This means t102 and all right-side temps are simultaneously live, increasing register pressure. An alternative: compute the right side first, *then* the left side. This is safe because all the expressions involved are pure (no side effects). Reordering reduces the number of simultaneously live temps.

Instruction Scheduling

A load instruction followed immediately by a use of the loaded value causes a **pipeline stall** (1–3 cycle latency). The fix: move independent instructions between the load and its use to fill the latency slot.

SAXPY example: The loop body of SAXPY (`y[i] = a * x[i] + y[i]`) has a chain of data dependencies. To schedule for a 2-wide processor, build a dependency graph of instructions, then at each cycle pick 2 ready instructions (ones whose inputs are available), preferring instructions on the **longest path** (critical path) to avoid delaying the overall schedule.

Loop unrolling: Copies the loop body multiple times within a single iteration. This creates more independent instructions that can be interleaved during scheduling. Requires alias analysis to prove that loads and stores from different iterations do not conflict. The C `restrict` keyword tells the compiler a pointer does not alias others. Limits: increased instruction-cache pressure, increased register pressure, and diminishing returns.

Liveness Analysis

What Does "Live" Mean?

A variable is **live** at a program point if its current value *might* be read in the future before being overwritten. A variable is **dead** if it will definitely be overwritten before it is read again (or is never read again).

Why this matters: Two variables that are live at the same time cannot share a register (they would overwrite each other). Liveness analysis tells us exactly which variables conflict, which is the foundation for register allocation.

Safety: It is safe to have **false positives** (saying a variable is live when it is actually dead)—this wastes a register but produces correct code. It is **unsafe** to have **false negatives** (saying a variable is dead when it is actually live)—this could allow overwriting a value that is still needed, breaking the program.

Precise liveness is undecidable in general (some control flow paths may be impossible to execute), so we conservatively approximate.

Basic Blocks

A **basic block** is a maximal sequence of instructions such that: you can only enter at the first instruction, you can only exit at the last instruction, and every instruction in between executes sequentially with no jumps in or out. A jump can only appear at the end of a block.

Def and Use Sets

For each basic block B, we compute:

- **def(B)**: The set of variables/temps that B writes to ("defines").
- **use(B)**: The set of variables/temps that B reads **before any write** to them within B.

The "before any write" part is critical: if a block writes `t100` and then later reads `t100`, the read uses the value that was just written *inside* the block, not an incoming value. So `t100` is in def(B) but not in use(B). Conversely, if a block reads `t100` and then later writes `t100`, the read needs the incoming value, so `t100` is in use(B).

Liveness Data Flow Equations

$$\text{live-in}(B) = \text{use}(B) \cup (\text{live-out}(B) - \text{def}(B))$$

$$\text{live-out}(B) = \bigcup_{S \in \text{succ}(B)} \text{live-in}(S)$$

Direction: Backward (information flows from uses back to definitions).
Meet operator: Union over successors (a variable is live-out if it is live-in at *any* successor).
Fixed point: Least (start with $\emptyset$ everywhere, only add variables).

In plain English: A variable is live coming into a block if either (a) the block uses it before writing it, or (b) it is live going out of the block and the block does not kill it (write to it). A variable is live going out of a block if any successor block needs it coming in.

Fixed Point Iteration Algorithm

1. **Initialize**: Set live-in(B) = $\emptyset$ and live-out(B) = $\emptyset$ for every block B.
2. **For each block** (preferring **reverse order** through the CFG for faster convergence), recompute live-in and live-out using the equations above.
3. **If anything changed** in any block's sets, repeat step 2.
4. **When nothing changes**: We have reached the fixed point. Done.

Why we must use the LEAST fixed point (starting from $\emptyset$): If we started with all variables assumed live (greatest fixed point), a variable in a loop would appear live everywhere simply because "it might be live." The least fixed point only marks a variable as live because there is an *actual use* that requires it. Starting from $\emptyset$ and growing ensures we only add variables that are genuinely needed.

Convergence guarantee: There are finitely many variables, and sets can only grow (we never remove from them). This monotonicity guarantees the iteration must converge.

Processing order: Working backward through the CFG converges faster because liveness information flows backward. But forward order also converges to the same correct answer—it just takes more iterations.

Worked Example: Instruction-Level Liveness

Consider a block with three instructions: `x = q + a; y = c * 3; z = 5`. The live-out of this block (from the successor blocks) is {x, y, u}.
We work **backwards** through the instructions:
After `z = 5`: z is defined but not in live-out, so no change. Remove z from live set (but it was not there). Live = {x, y, u}.
After `y = c * 3`: y is defined (remove from live set). c is used (add to live set). Live = {x, c, u}.
After `x = q + a`: x is defined (remove from live set). q and a are used (add to live set). Live = {q, a, c, u}.
The live-in of this block is {q, a, c, u}. Each instruction: remove the destination (kill), add the sources (gen).

Worked Example: Liveness with Loops (Vampire/Zombie Example)

This example has a CFG with loops and variables named vampires, zombies, x, y, z, buffy, answer, snack.
First pass (working backward): Some blocks' successors have not been processed yet (their sets are still $\emptyset$), so we get partial results. Variables that propagate through the loop back-edges are missed.
Second pass: Now the successor blocks have non-empty live-in sets from the first pass. Variables used inside the loop (like "zombies") propagate backward through the back-edges. "Zombies propagate everywhere—which is what zombies do."
Third pass: Nothing changes. We have converged.
Key insight: Liveness information propagates backward through loop back-edges. The first pass discovers some liveness information; the second pass propagates it through cycles in the CFG; the third pass confirms convergence.
Only-add property: We never remove variables from liveness sets during iteration. Once a variable enters a set, it stays. This monotonicity guarantees convergence.

Infeasible Paths

If `x < 7` at one branch implies `x < 9` at a later branch, the path where `x < 7` is true but `x >= 9` is impossible. Standard liveness analysis cannot detect this—it considers all CFG paths, including infeasible ones, leading to false positives. Adding more sophisticated analysis improves precision but can never be perfect (the general problem is undecidable). "The Turing cliff is always there"—trying to be perfect makes your analyzer loop forever.

View Shift (procEntryExit)

The calling convention says arguments arrive in a0–a3 (and on the stack for the 5th argument and beyond). But the function's IR expects variables in specific temps or frame slots. The **view shift** bridges this gap by inserting moves at the function entry:

For example: `MOVE(temp100, a0)` (parameter x from register), `SW a1, -4(fp)` (parameter y escapes, store to frame), `LW temp103, ??(sp)` (5th parameter b from stack).

Register-to-register moves from the view shift might be eliminated later if the register allocator happens to assign the source and destination to the same physical register.

procEntryExit has three phases:

- **procEntryExit1**: View shift (move arguments to where the IR expects them) and save/restore callee-saved registers.
- **procEntryExit2**: "Sink instruction"—artificially defines all callee-saved registers and the return-value register as live at the end of the function. This prevents the register allocator from using those registers for other values that would be lost at function exit.
- **procEntryExit3**: Generates the actual assembly prologue (function label, SP adjustment) and epilogue (`JR RA` with `SOME([])`).

Register Allocation

The Interference Graph

An interference graph has one **node** for each temp/variable in the program. Two types of edges:

- **Solid edge (interference)**: Two temps are live at the same time somewhere in the program. They *cannot* be assigned the same register.
- **Dashed edge (move-related)**: A move instruction copies one temp to the other. We would *like* them in the same register so the move can be eliminated.

Building the graph: For each instruction that writes to temp t, add interference edges between t and every variable that is **live-out** after that instruction (except t itself). For MOVE instructions, add a move edge (not interference) between source and destination.

Let k = number of physical registers (about 25 for MIPS). The goal is to **k-color the graph**: assign each node a color (register) such that no two adjacent nodes share the same color.

Graph Coloring is NP-Complete

NP stands for **nondeterministic polynomial**—it means a proposed solution can be *verified* in polynomial time, not that the problem cannot be solved in polynomial time. Whether P = NP is an open $1 million problem.

The best known exact algorithm for graph coloring is exponential. For typical programs and ∼25 MIPS registers, brute force would technically work, but we use a fast heuristic instead.

Safe approximation: Our heuristic may report "cannot k-color" when a valid coloring actually exists. This results in an unnecessary spill (storing a variable in memory instead of a register)—suboptimal but correct. What we must *never* do is assign the same color

to two adjacent nodes, as that would produce incorrect code.

Trivial Degree

A node has **trivial degree** if its degree (count of **interference edges only**—move edges do not count) is strictly less than k.

Key property: If the rest of the graph (without this node) can be k-colored, a trivial-degree node can *always* be colored. It has fewer than k neighbors, so at least one color is unused by its neighbors and available for it.

The Algorithm: Simplify → Coalesce → Freeze → Spill

Phase 1—SIMPLIFY: Find a node that (a) has trivial degree ($< k$ interference edges) and (b) is **not move-related**. Remove it from the graph and push it onto a stack (recording its adjacencies). Removing a node may lower its neighbors' degrees, potentially creating new trivial-degree nodes. Repeat until no eligible node remains.

We skip move-related nodes because we want to try coalescing them first (to eliminate moves).

Phase 2—COALESCE: Try to merge two move-related nodes into one, eliminating the register-to-register move. Only do this if it is safe (will not make the graph harder to color). Two heuristics determine safety:

Coalescing Heuristics (if EITHER says yes ⇒ safe to coalesce)

Briggs heuristic: Count how many neighbors of the *combined* node have non-trivial degree ($\geq k$). If this count is $< k$, coalescing is safe. The reasoning: all trivial-degree neighbors can be simplified away later, so only non-trivial ones pose a risk. If fewer than k are non-trivial, the combined node will eventually become simplifiable.
George heuristic (merge A into B; **asymmetric**): For every neighbor T of A, check: either T **already interferes with B** (merging A into B adds no new constraint for T), or T has **trivial degree** (T can be simplified anyway). If all of A's neighbors pass this check, coalescing is safe. **Important**: Check both directions (A into B and B into A) separately—they may give different answers.
If **either** heuristic says yes in **either** direction, the coalesce is safe. Proceed.

After a successful coalesce, go back to SIMPLIFY (merging may have opened new simplification opportunities).

Phase 3—FREEZE: If we are stuck (cannot simplify or coalesce anything), pick a move edge and remove it (give up on that move elimination). This makes the previously move-related nodes eligible for simplification. Prefer to freeze edges where at least one endpoint has trivial degree. Then go back to SIMPLIFY.

Phase 4—POTENTIAL SPILL: If still stuck (cannot simplify, coalesce, or freeze), remove an arbitrary node from the graph and push it onto the stack, marked with a star ("potential spill"). When we pop it later, it might not be colorable. Heuristic for which node to spill: low-degree nodes are more likely to get lucky; high-degree nodes remove more constraints.

Phase 5—SELECT (Pop and Color): Pop nodes from the stack one at a time and assign each the lowest available color not used by its already-colored neighbors. Nodes removed during SIMPLIFY are *guaranteed* to get a color (by the trivial-degree property). Potential-spill nodes may fail to find a color → this becomes an **actual spill**: insert load/store instructions to/from the stack for every access, creating very short live ranges, and **restart the entire algorithm from scratch**.

Important: A failure to color does NOT prove the graph is not k-colorable—it only means this particular ordering of nodes failed. A different ordering might succeed. Full backtracking would guarantee correctness but would be exponentially slow.

Worked Example: Coalescing Check for C and D

Checking whether nodes C and D (connected by a move edge) can be coalesced. $k = 3$.
George, merging C into D: Check each neighbor of C:

- M: Does M already interfere with D? **YES** ✓ (no new constraint added)
- B: Does B already interfere with D? **YES** ✓

All of C's neighbors pass → George says **safe**.
George, merging D into C: Check each neighbor of D:

- M, B: pass ✓
- K: Does K interfere with C? **NO**. Is K trivial degree? **NO** → **FAILS**

This direction fails.
Result: One direction passed, so we proceed with the coalesce.
Why George is asymmetric: Merging A into B only adds A's neighbors as new neighbors of B. If all of A's neighbors are already connected to B or are trivially removable, the merge does not increase effective degree.

Worked Example: Coalescing Check for B and J

Checking whether B and J can be coalesced. $k = 3$.
Briggs: The combined BJ node's neighbors of non-trivial degree (≥ 3): E (non-trivial), M (non-trivial), CD (non-trivial) = 3 non-trivial neighbors. Since $3 \geq k = 3$, Briggs says **UNSAFE**.
George, B into J: B's neighbors include M. Does M interfere with J? **NO**. Is M trivial degree? **NO** → **FAILS**.
George, J into B: J's neighbors include F. Does F interfere with B? **NO**. Is F trivial degree? **NO** → **FAILS**.
Result: Neither heuristic says safe in either direction → **cannot coalesce B and J**.

Practice Final Q4—3-Register Machine

Nodes: x, y, z, a, b, c, p, q. $k = 3$.
Step 1—Simplify: b has trivial degree (< 3 interference edges) and is not move-related. Remove b, push onto stack.
Step 2—Coalesce: p and q are move-related. Check safety with heuristics: safe. Merge into combined node "pq".
Step 3—Simplify: pq now has degree < 3. Remove and push.
Step 4—Freeze: x and c are connected by a move edge but cannot be safely coalesced (combined node would have too many high-degree neighbors). Give up: remove the move edge.
Step 5—Simplify: Now x, y, a, z, c all have trivial degree or are no longer move-related. Remove and push each.
Pop and assign colors:
c = r1, z = r2, a = r3, y = r1, x = r2, pq = r2, b = r3.
Since p and q were coalesced into pq, they both get register r2. The move instruction between them is **eliminated**—no copy needed.

Pre-colored Temps

Some temps *must* be assigned to specific physical registers (e.g., `temp100` must be a0 because of the calling convention). All pre-colored temps interfere with each other. They are **never simplified or spilled**—they serve as the base case of the algorithm (when only pre-colored nodes remain, the graph is trivially colored).

Automatic caller/callee-saved assignment: A temp that is live across a JAL instruction interferes with *all* caller-saved registers (because the call may overwrite them). This naturally forces the allocator to assign it a callee-saved (S) register. Short-lived temps near calls can use caller-saved (T) registers. The graph coloring algorithm handles this automatically.

S-register saving strategy: Try allocating with 0 callee-saved registers first. If allocation fails, try with 1 (s0 available), then 2, etc. This avoids wastefully saving registers that are not needed. When assigning colors, use the *lowest-numbered* color first to promote register reuse and leave more registers available.

Spilling

If coloring fails for a node: mark that variable as "spilled." Every access to that variable becomes a load from the stack (before reads) and a store to the stack (after writes). This creates very short live ranges with minimal interference. Then **restart the entire register allocation from scratch** with the new code.

If it fails again, spill another variable. More sophisticated approaches: analyze where register pressure is highest and add loads/stores only at those points. Consider loop context: if the spilled variable is used inside a loop, save it before the loop and load it after.

Data Flow Analysis Framework

Data flow analysis computes information about what *might* or *must* be true at each point in a program. Different analyses answer different questions, but they all follow the same framework: define equations over sets, initialize them, and iterate until convergence (the fixed point).

Master Table of Data Flow Analyses

Analysis	Direction	Meet Op	Init	Fixed Point	Drives Optimization
Available Expressions	Forward	∩ (intersection)	Full set	Greatest	CSE
Reaching Definitions	Forward	∪ (union)	∅	Least	Constant/copy propagation
Liveness	Backward	∪ (union)	∅	Least	Register allocation
Very Busy Exprs	Backward	∩ (intersection)	Full set	Greatest	Code hoisting

The rule: Intersection (∩) ⇒ greatest fixed point (start with the full set, shrink toward the answer). Union (∪) ⇒ least fixed point (start with ∅, grow toward the answer).
Intuition: Intersection analyses are "innocent until proven guilty"—assume everything is true until you find a path that contradicts it. Union analyses are "guilty until proven innocent"—assume nothing until you find evidence.

Available Expressions

An expression x op y is **available** at a program point if it has been computed on **every path** reaching that point, and neither x nor y has been modified since the computation. If an expression is available, we can reuse the previously computed value instead of recomputing it.

$$\text{AE-in}(B) = \bigcap_{P \in \text{pred}(B)} \text{AE-out}(P)$$

$$\text{AE-out}(B) = (\text{AE-in}(B) - \text{Kill}(B)) \cup \text{Gen}(B)$$

Gen(B): Expressions computed in B whose operands are not redefined in B after the computation.

Kill(B): Any expression containing a variable that B writes to. Also: a write to memory kills any expression involving a memory read (unless alias analysis can prove they access different locations).

Worked Example: Why Available Expressions Needs the Greatest Fixed Point

Block 1 computes `a = x * y`. Then there is a loop (which does not modify x or y). After the loop, Block 2 uses `b = x * y`.
Wrong approach (start with ∅, least fixed point): At the loop's merge point, we intersect $\{x * y\}$ (from Block 1) with ∅ (from the loop back-edge, still uninitialized). The intersection is ∅. So $x * y$ is never marked as available at Block 2. **This is wrong**—we should be able to reuse the computation.
Correct approach (start with full set, greatest fixed point): At the merge point, we intersect $\{x * y\}$ (from Block 1) with the full set (from the loop back-edge's initial value). The intersection is $\{x * y\}$. After one more iteration, the loop body's output replaces the full set with the actual expressions, and $x * y$ survives because the loop does not modify x or y. $x * y$ is correctly marked as available at Block 2.
General principle: Cycles with intersection need the greatest fixed point. Starting with ∅ kills everything through cycles. Starting with the full set lets correct information survive until contradicted.

How CSE (Common Subexpression Elimination) Actually Works

If $x * y$ is available at a point via *different* computations along different paths (e.g., `a = x*y` on one path and `b = x*y` on another), we cannot simply use `a` or `b` at the use point (their values may have changed). Solution: at each original computation site, insert an extra assignment `temp = result`. At the reuse point, use `temp`. This may introduce reg-to-reg moves (but those are cheaper than a multiply and may be eliminated by the register allocator).

CSE tradeoff—live range extension: CSE keeps the computed value alive in a temp for later reuse. This extends the temp's live range, possibly across the entire program, which **increases register pressure** and may force spills. Sometimes **re-computing the expression is cheaper than spilling a register**. This is a real tradeoff the compiler must navigate.

Reaching Definitions

A definition (an assignment like "`x := ...`" at line L) **reaches** a use of x if there exists at least one path from L to the use with no other definition of x along that path.

$$\text{RD-in}(B) = \bigcup_{P \in \text{pred}(B)} \text{RD-out}(P)$$

$$\text{RD-out}(B) = (\text{RD-in}(B) - \text{Kill}(B)) \cup \text{Gen}(B)$$

Gen(B): The definitions made in B (specifically, the *last* definition of each variable within B).

Kill(B): All *other* definitions of the same variables anywhere else in the program.

Note: "definition" means an assignment statement, not a variable declaration. Two assignments to x at different lines are two different definitions.

Worked Example: How Reaching Definitions Drive Optimizations

1. Constant Propagation: If the only definition reaching a use of x is `x = 0`, and we encounter `a = x + q`, we can replace x with 0: `a = 0 + q = q`. Then constant folding simplifies further.
2. Copy/Move Propagation: If the only reaching definition is `x = y`, replace all uses of x with y. The assignment `x = y` then becomes dead code and can be deleted.
3. Dead Code Elimination: A definition that reaches *no* uses (and has no side effects like I/O) is useless. Delete it.
4. Uninitialized Variable Detection: If no definitions reach a particular use, the variable may be uninitialized. The compiler can issue a warning.
5. Live Range Splitting: See below.

Practice Final Q5—Reaching Definitions

CFG: Block A (instructions 1–3) → Block B (instructions 4–7) → either Block C (8–10) or Block D (11–14) → back to B or to Block E (15 = return).
Which definitions of a reach the use of a in instruction 4? Answer: instructions **1 and 8**. Definition at instruction 1 (in Block A) flows through to B. Definition at instruction 8 (in Block C) flows back through the loop to reach B.
Which definitions of a reach the use of a in instruction 8? Answer: instructions **1, 8, and 13**. From A via B (def at 1). From C looping back (def at 8). From D's definition at instruction 13, going back to B, then to C.
Which definitions of b reach the use of b in instruction 6? Answer: instructions **2 and 11**. Definition at 2 from Block A, and definition at 11 from Block D looping back through B.

Live Range Splitting

Label all definitions of a variable (D0, D1, ...) and all uses (A, B, ...). For each use, determine which definitions reach it using reaching definitions analysis. Build a graph connecting each use to its reaching definitions. Each **connected component** of this graph represents a separate "live range"—a portion of the program where the variable carries one particular value. Rename each component as a separate temp. This reduces interference because the distinct live ranges no longer conflict with each other.

Example: Two for-loops both use loop variable `i`. Definitions D0, D1 (in loop 1) reach uses A–D. Definitions D2, D3 (in loop 2) reach uses E–H. These form two disjoint connected components → split into `i` and `i'`. Now these two temps do not interfere, making register allocation easier.

Very Busy Expressions

An expression is **very busy** at a program point if it will **definitely be computed** on **all forward paths** from that point before any of its operands are modified.

$$\text{VBE-in}(B) = (\text{VBE-out}(B) - \text{Kill}(B)) \cup \text{Gen}(B)$$

$$\text{VBE-out}(B) = \bigcap_{S \in \text{succ}(B)} \text{VBE-in}(S)$$

This is a backward analysis with intersection, so it uses the greatest fixed point (start with full set).

What it drives: **Code hoisting**—if an expression is very busy (will definitely be computed on all paths), we can compute it earlier (at the current point) to reduce code duplication. This helps with I-cache (less duplicated code) and with LICM (a very busy expression inside a loop body is guaranteed to execute, making it safe to hoist).

Exception behavior warning: Moving an expression like `x / y` earlier could cause a divide-by-zero exception to happen at a different point than the programmer expects. This can be confusing during debugging.

Practice Final Q5—Matching Analyses to Optimizations

Common Subexpression Elimination (CSE)	Available Expressions
Forward flow with union meet	Reaching Definitions
Def/Use web formation (live range splitting)	Reaching Definitions
Backwards flow with intersection meet	Very Busy Expressions

Practice Final Q5—Very Busy Expressions Table

Expressions: a+1, m-1, a+b, b*47, x+y, b+1, arr[b].

Point	a+1	m-1	a+b	b*47	x+y	b+1	arr[b]
After block 3	Y	N	Y	Y	N	N	N
After block 7	Y	Y	N	N	N	N	N
After block 10	N	N	N	N	N	N	N
After block 14	Y	Y	N	N	N	N	N
After block 15	N	N	N	N	N	N	N

After block 3: a+1 is very busy because both branches (C and D) compute a+1 before modifying a. a+b is very busy because instruction 4 computes it on all paths. b*47 is very busy because instruction 6 computes it on all paths. After block 15 (return): all N—nothing is computed after the program ends.

Compiler Optimizations (Comprehensive)

Worked Example: Optimizing Quicksort Partition

The naive code for quicksort's partition function computes `4*i` five times, `4*j` multiple times, and `4*high` repeatedly.

1. Strength Reduction (during instruction selection): Replace `4*i` with `i << 2` (left shift by 2). Multiplication takes 3–4 cycles; a shift takes 1 cycle.

2. Common Subexpression Elimination (using Available Expressions): Compute `4*i` once as `t2` in block B2. Is `4*i` still available at other uses? Check: is i modified between B2 and those uses? No → reuse `t2` everywhere. This eliminates 4 redundant multiplications. Apply the same reasoning to `4*j` (reuse `t4`) and `4*high` (reuse `t1`).

3. Copy Propagation and Dead Code Elimination (using Reaching Definitions): After CSE, we have `t6 = t2` (which was `4*i`). The only reaching definition of `t6` is this copy. Replace all uses of `t6` with `t2`. Now `t6 = t2` has no uses → delete it (dead code).

4. Reusing Loads: `a[t2]` was loaded into `t3` in block B2. If there are no stores between B2 and B5, `t3` still holds the correct value—reuse it. The swap in B5 goes from 9 instructions down to 3 (just the stores and a branch).

5. Alias Analysis Gotcha: We cannot reuse `a[t1]` (the value at `a[high]`) in block B6 after the swap in B5, because B5 writes to `a[t2]` and `a[t4]`, which *might* be the same address as `a[t1]`. The compiler cannot prove they are different (alias analysis fails), so it must re-load.

6. Induction Variable Elimination: The variable i only changes as $i = i + 1$, and is only used as $4 * i$. Introduce a new variable $t2$ that starts at $4 * i_0$ and increments by 4 each iteration. Replace all uses of $4 * i$ with $t2$. Eliminate i entirely. Similarly, $j \rightarrow t4$ decremented by 4 each iteration.

7. Comparison Replacement: Replace the comparison `i >= j` with `t2 >= t4`. **Warning**: $4i \geq 4j$ is not always equivalent to $i \geq j$ in fixed-width arithmetic due to overflow! But since $4i$ is an array index, overflow would cause a segfault before reaching this comparison, so the transformation is safe here. "Something to be wary of in general."

Optimization Summary: Every Optimization Explained

Common Subexpression Elimination (CSE): If an expression has already been computed and its operands have not changed, reuse the previous result instead of recomputing. Driven by **Available Expressions** analysis. Tradeoff: extends live ranges, increasing register pressure.

Constant Propagation: If the only reaching definition of a variable is a constant assignment (e.g., `x = 5`), replace all uses of that variable with the constant. Driven by **Reaching Definitions**. Enables further constant folding (e.g., `5 + 3` becomes `8`).

Copy Propagation: If the only reaching definition is a copy (e.g., `x = y`), replace all uses of `x` with `y`. Driven by **Reaching Definitions**. The copy then becomes dead code.

Dead Code Elimination: A definition whose result is never used (reaches no uses) and has no side effects can be deleted. Driven by **Reaching Definitions** and **Liveness** (a variable that is not live after its definition is dead).

Strength Reduction: Replace expensive operations with cheaper ones. Multiplication by a power of 2 becomes a left shift. Multiplication by small constants becomes shift + add/subtract combinations. Driven by pattern matching during instruction selection.

Induction Variable Elimination: A variable that changes linearly with the loop counter (e.g., `4*i` where `i` increments by 1) can be replaced with a new variable that increments by the stride (4) each iteration, eliminating the multiplication entirely. Detected by identifying **back edges** (loop detection) and analyzing how variables change each iteration.

Loop Invariant Code Motion (LICM): An expression inside a loop whose operands do not change within the loop can be computed once before the loop. Detected using **Reaching Definitions** (all reaching definitions come from outside the loop) and **Domination** (to determine loop structure). **Danger**: If the loop might execute zero times, hoisting moves code that would never have executed—potentially causing crashes (e.g., null pointer dereference). Fix: convert to a `do-while` with an `if` guard, ensuring the body executes at least once. Use **Very Busy Expressions** to verify the expression is guaranteed to execute.

Code Hoisting: If an expression is very busy at a point (will definitely be computed on all forward paths), compute it once at that point instead of redundantly in each branch. Driven by **Very Busy Expressions**. Reduces code size and improves I-cache behavior.

Live Range Splitting: Split a single variable into multiple independent temps based on which definitions reach which uses. Driven by **Reaching Definitions**. Each connected component in the def-use graph becomes a separate temp, reducing interference in the register allocation graph.

Optimization Pass Ordering

There is **no single correct ordering** of optimization passes. Optimizations can enable or conflict with each other:

- CSE extends live ranges → makes register allocation harder.
- Inlining a function enables more CSE (newly visible redundancies).
- Copy propagation creates dead code → dead code elimination cleans it up.
- Constant propagation feeds constant folding, which may enable further propagation.

GCC runs three separate instruction scheduling passes at different stages. Iterating all passes multiple times can help but shows diminishing returns.

Worked Example: LICM Danger—Zero-Trip Loops

Code: `for(i) for(j in 0..m) total += a[i][j]`
The expression `a[i]` is loop-invariant with respect to the inner loop (only `j` changes). We want to hoist `p = a[i]` before the inner loop.
The danger: If `m = 0` and `a = null`, the original code never enters the inner loop body, so `a[i]` is never evaluated—no crash. But if we hoist `p = a[i]` before the loop, it executes even when m = 0, causing a **segfault** in previously working code.
The fix: Convert the while-loop to `if (m > 0) { do { ... } while (...); }`. The `if` guard handles the zero-trip case. Inside the do-while, the body executes at least once, so `a[i]` is a **very busy expression** (guaranteed to execute on all paths through the loop). Now hoisting is safe.
This is a key application of Very Busy Expressions analysis.

Domination and Loop Identification

What is Domination?

Node A **dominates** node B (written $A \in \text{Dom}(B)$) if **every path** from the program start to B must pass through A. Every node dominates itself (by convention).

The dominator equation:

$$\text{Dom}(B) = \{B\} \cup \bigcap_{P \in \text{pred}(B)} \text{Dom}(P)$$

For the entry node: Dom(entry) = {entry}. All other nodes are initialized to the universal set (all nodes). This is a forward analysis with intersection, so it uses the greatest fixed point.

Immediate dominator (idom): The closest strict dominator of B (the dominator of B that is not B itself and is dominated by all other dominators of B). The dominator tree has an edge from idom(B) to B. The path from the root to B in this tree lists all of B's dominators.

Back Edges and Natural Loops

A **back edge** is an edge from node E to node S where S **dominates** E. Each back edge defines a **natural loop**: the set of nodes that can reach E without going through S, plus S itself. The node S is the loop **header**—the single entry point.

Not every cycle is a natural loop: If two nodes have edges in both directions but neither dominates the other, this is not a natural loop. It has no clear single entry point, which typically arises from arbitrary `goto` statements rather than structured loops.

The loop header dominates every node in the loop body—you cannot enter the loop without passing through the header.

Practice Final Q6—Domination (10-Node CFG)

CFG edges: 0 → {1,2}, 1 → 4, 2 → 3, 3 → {3,4}, 4 → {1,5}, 5 → {6,7}, 6 → 8, 7 → 8, 8 → {4,9}.

Dominator sets (computed by iterating the equation):

Dom(0) = {0}	Dom(5) = {0, 4, 5}
Dom(1) = {0, 1}	Dom(6) = {0, 4, 5, 6}
Dom(2) = {0, 2}	Dom(7) = {0, 4, 5, 7}
Dom(3) = {0, 2, 3}	Dom(8) = {0, 4, 5, 8}
Dom(4) = {0, 4}	Dom(9) = {0, 4, 5, 8, 9}

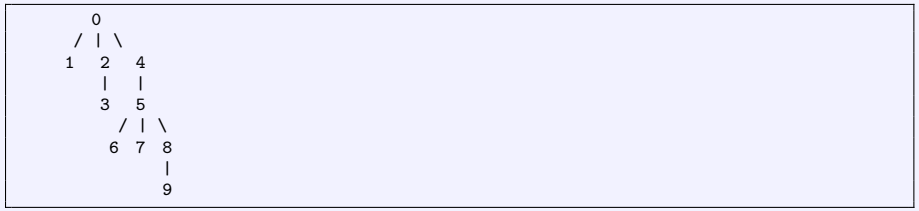
Key reasoning for Dom(4): Two paths reach node 4: 0 → 1 → 4 and 0 → 2 → 3 → 4. Node 1 is on the first path but not the second. Nodes 2 and 3 are on the second but not the first. Only node 0 (and 4 itself) appear on *every* path to 4.

Back edges (which define loops):

- 3 → 3: This is a self-loop. Node 3 dominates itself (trivially). Loop body = {3}.
- 8 → 4: Node 4 dominates node 8 (check: 4 ∈ Dom(8)? Yes). Loop body = {4, 5, 6, 7, 8}.

4 → 1 **is NOT a loop**: Does node 1 dominate node 4? Check: 1 ∈ Dom(4) = {0, 4}? **No**. The path 0 → 2 → 3 → 4 bypasses node 1 entirely. So this edge looks like it might form a loop, but it fails the domination test.

Immediate dominator tree:



idom(1) = 0, idom(2) = 0, idom(4) = 0, idom(3) = 2, idom(5) = 4, idom(6) = 5, idom(7) = 5, idom(8) = 5, idom(9) = 8.

Garbage Collection

What is garbage? Memory that is **unreachable** from any variable the program can access. This is a safe approximation of "memory that can't affect future computation" (which is undecidable).

Reference Counting

Each object maintains a count of how many pointers point to it. When the count drops to 0, the object is garbage and can be freed (recursively decrementing counts for any objects it points to).

Worked Example: Code Generated for p

A simple pointer assignment `p = q` expands to this:

```
// Step 1: Increment q's refcount FIRST
if (q != null) {
    temp = q->refcount;
    temp++;
    q->refcount = temp;
}
// Step 2: Decrement p's refcount
if (p != null) {
    temp = p->refcount;
    temp--;
    if (temp == 0)
        recursive_free(p); // free p and everything it points to
    else
        p->refcount = temp;
}
// Step 3: Do the actual pointer assignment
p = q;
```

What was a single register-to-register move becomes approximately 3 branches, 2 loads, 2 stores, 2 arithmetic operations, and 1 move. "And a partridge in a pear tree."

In multithreaded code, the refcount updates need to be atomic operations (∼50 cycles each instead of 1 cycle), making the overhead even worse.

CRITICAL: You must increment q's refcount *before* decrementing p's. If p and q point to the *same* object and you decrement first, the refcount hits 0 and the object is freed—before you can increment it back!

Pros: Conceptually simple. Incremental (no "stop the world" pauses).

Cons: Very slow (approximately 20–30% runtime overhead). **Cannot collect cycles**: if A points to B and B points to A, both have refcount ≥ 1 even if nothing else points to either. 1 extra word per object for the refcount. Python uses reference counting as its primary GC.

Mark and Sweep

Root set: All pointers in registers, on the stack, and in global variables. These are the starting points—anything reachable from these is live.

Mark phase: Perform a depth-first search from the root set. Mark every reachable object (using a mark bit in the object header, which also serves as the "visited" set for the DFS—this correctly handles cycles).

Sweep phase: Walk through the **entire heap linearly** using size headers to step from one object to the next. Unmarked objects → add to the free list. Marked objects → clear the mark bit (for next collection).

Cost: O(live + dead)—must touch *every* object during the sweep, even dead ones. Must **stop the world** (pause program execution) during collection.

Space overhead: 1 word per object (for the size header and mark bit). Causes **fragmentation**: freed objects leave holes of various sizes in the heap.

Making the heap bigger does not help much: both the amount of live data and the amount of dead data grow proportionally, so the amortized cost per collection stays roughly the same.

How the GC Identifies Pointers

The GC needs to know which fields of each object are pointers (to follow during marking) versus plain integers (to ignore). Three approaches:

1. **Compiler-generated type information (bitmasks)**: The compiler attaches a bitmask to each object type (1 bit per field: 1 = pointer, 0 = not). Stack frames have similar bitmasks. For registers, a lookup table keyed by the program counter (return address) maps to a 32-bit mask of which registers hold pointers at that point.
2. **Conservative collection**: Treat any value that looks like a valid heap address as a potential pointer. **Downside**: Cannot move objects, because a non-pointer integer that happens to look like a heap address would become invalid if the "target" object moved.
3. **Tagging**: Steal the low bit of every word. Since pointers are word-aligned (low bits are 0), use 0 = pointer and 1 = integer. **Downside**: Integers lose 1 bit of range. Every arithmetic operation needs extra mask/shift operations.

Stop and Copy

Worked Example: The Stop-and-Copy Algorithm

The heap is split into two halves: **active** (where allocation happens) and **inactive** (empty). Allocation in the active half uses a **bump pointer**—just increment a pointer to allocate, as fast as stack allocation.
When the active half fills up:

1. Start a DFS from the root set.
2. For each live object encountered, **copy** it to the inactive half.
3. Leave a **forwarding pointer** in the old location (overwriting the object).
4. When you encounter a reference to an already-copied object (it has a forwarding pointer instead of normal data), update the reference to point to the new copy.
5. Dead objects are **never touched**—they are simply overwritten when the halves swap.

Concrete example: Root set points to D. Copy D to inactive half, leave forwarding pointer. D references B → copy B, leave forwarding pointer. B references A → copy A. Later, G references B → B already copied (has forwarding pointer) → update G's pointer to B's new address. Objects C, E, F are dead—never touched, never copied, cost nothing.
After copying all live objects, swap the roles of the two halves. The old active half is now meaningless bits.

Pros: Cost is O(live) only—dead objects cost absolutely nothing. No fragmentation (compaction is automatic, since objects are copied consecutively). Extremely fast allocation (bump pointer). Good locality (objects that reference each other are copied next to each other).

Cons: **Wastes half the heap** (only one half is usable at a time). Has copying overhead (but most objects are small). All objects change addresses, so every pointer in the program must be updated.

Generational Garbage Collection

Key insight ("infant mortality"): The vast majority of objects die very young. Objects that survive several collections tend to live for a long time. This is true across almost all programs.

Multiple generations:

- **Generation 0 (nursery)**: Small (∼256KB, sized to fit in L2 cache). Uses stop-and-copy. Collected very frequently. About 95% of its contents are garbage each time → collection is extremely fast.
- **Generation 1**: Medium (∼4MB). Typically uses mark-and-sweep. Fills up slowly because only survivors from Gen 0 are promoted here.
- **Generation 2**: Large (∼6GB). Collected rarely.

Objects start in Gen 0. After surviving N collections (tracked by a 2-bit counter in the object header, typically surviving 3 times), they are promoted to Gen 1. Similarly from Gen 1 to Gen 2.

Why size Gen 0 to fit L2 cache: If Gen 0 is 256KB and L2 cache is 512KB, all the copying during Gen 0 collection hits the cache. No expensive main-memory accesses.

The "old points to young" problem: An old-generation object might point to a young-generation object (due to mutation). When collecting the young generation, we need to find these cross-generational pointers (they are roots for the young-gen collection). But scanning all old-gen objects would defeat the purpose of generational GC.

Solutions:

- **Purely functional languages** (like Haskell): Objects can only point to older objects (no mutation). The problem cannot occur.
- **Imperative languages**: Use `mprotect()` to mark old-generation pages as read-only. Any write triggers a SIGSEGV signal. The signal handler records which page was written to ("dirty page") and marks it writable again. During young-gen collection, only dirty pages need to be scanned for cross-generational pointers.

Promoting referenced objects: When moving object D to an older generation, also promote the objects D references (like B and A). They are likely long-lived too (they have been kept alive by D).

Java's permanent generation: Class objects, method metadata, and other JVM internals that essentially never die are placed in a special region that is never garbage-collected.

Practice Final Q7—Garbage Collection

Name 3 advantages of garbage collection: (1) Fast allocation with stop-and-copy (bump pointer—almost as fast as stack allocation). (2) Improved memory locality with stop-and-copy (objects that reference each other end up adjacent). (3) No dangling pointer bugs (use-after-free is impossible). (4) No double-free bugs. (5) No memory leaks (unreachable memory is automatically reclaimed). (6) Reduced development and debugging time.
Note: If an advantage is specific to one GC algorithm (like "fast allocation" for stop-and-copy), you must say which algorithm.
Tradeoffs of each algorithm:

- **Reference counting**: Very high per-operation overhead (∼20–30%), 1 word per object, cannot collect cycles.
- **Mark-and-sweep**: O(live + dead) cost, 1 word per object, causes fragmentation, requires stopping the world.
- **Stop-and-copy**: O(live) cost, **wastes half the heap**, compacts automatically, extremely fast allocation.

Three methods for identifying pointers: (1) Compiler-generated type info / bitmasks (most precise). (2) Conservative collection (treats anything that looks like a pointer as one; cannot move objects). (3) Tagging (steal 1 bit per word; lose 1 bit of integer range).

Compiling Object-Oriented Languages

Types of Polymorphism

1. **Ad-hoc polymorphism (overloading)**: Same function name, different implementations for different types. In C++, the compiler uses **name mangling** to create unique names: `add_int_int` vs. `add_float_float`. Resolved at compile time.
2. **Parametric polymorphism**: Code works for any type. C++ templates: specialized per type (separate machine code for each). Java generics: compiled once using type erasure (all type parameters become `Object`; type parameter not available at runtime, so you cannot write `new T()`).
3. **Subtype polymorphism**: A function accepting a superclass also accepts any subclass. Implemented via vtables and dynamic dispatch.

VTables and Dynamic Dispatch

Naive approach: put function pointers directly in every object. Space cost = (number of methods) × (number of objects). Too expensive.

Better approach: Each object stores **one pointer** to a shared **vtable** (virtual method table). All instances of the same class share one vtable. Space cost = (number of objects) + (number of methods). Much better.

Worked Example: How Dynamic Dispatch Works

Given the call `p->speak()`, the generated code is:

```
v = load_word(p, 0)      // load vtable pointer from object
f = load_word(v, offset) // load method pointer from vtable
a0 = p                   // pass 'this' as implicit first argument
jalr f                   // indirect call through function pointer
```

Cost vs. static dispatch: Two extra load instructions per call (vtable pointer + method pointer). For a static (non-virtual) call, you would just `jal speak` directly.
Why C++ requires the `virtual` keyword: In the 1980s on 12 MHz processors, two extra memory loads per method call were a significant cost. So C++ made dynamic dispatch opt-in: only methods declared `virtual` use vtables. **The danger**: If you forget `virtual` on a base class method but "override" it in a derived class, C++ silently uses static dispatch—the derived class's version is never called. This is a subtle, common bug.
Why Java makes everything virtual: By the 1990s, hardware was fast enough that the overhead was negligible. And 99.9% of the time, a method override that is not dispatched dynamically is a bug. Java removes the footgun by making all methods virtual by default.

Single Inheritance and the Sub-object Rule

A subclass must contain its superclass's fields at the **same memory offsets**. For example, if Animal has fields `[vtable | name | age]`, then Cat (which extends Animal) has `[vtable | name | age | laziness]`. The first three fields form a valid Animal sub-object at identical offsets.

This means code compiled for Animal that accesses `p->name` as `load(p, 4)` works correctly whether `p` actually points to an Animal or a Cat. Cat's vtable has the same layout as Animal's for inherited methods; overridden methods point to Cat's implementation.

Multiple Inheritance

When a class inherits from two parents, the sub-objects are stacked: `[cat_vtable | name | age | laziness | robot_vtable | robot_name | model]`. Note: **two vtable pointers**.

Primary parent (Cat, listed first): Casting to Cat or Animal is free—the pointer does not change.

Non-primary parent (Robot): Casting requires **pointer adjustment**: add an offset so the pointer points to the Robot sub-object within the full CyborgCat object. The call becomes `f(p + offset)` instead of `f(p)`.

Thunks: If CyborgCat overrides a Robot method M2, Robot's vtable entry for M2 cannot point directly to CyborgCat's code (because the `this` pointer would be pointing at the Robot sub-object, not the full CyborgCat). Instead, it points to a small **thunk**: `this -= 24; jump to actual_M2`. The thunk adjusts the `this` pointer before calling the real method. Non-overridden methods do not need thunks—they expect the Robot sub-object pointer, which is correct.

The Diamond Problem and Virtual Inheritance

If two parent classes share a common grandparent, the grandparent's fields appear twice (one copy in each parent's sub-object). This wastes space and causes ambiguity.

C++ solution: **virtual inheritance**. The shared base class is placed at the **end** of the object (not inside either parent). Only one copy of the shared fields exists. Accessing those fields requires an **extra indirection through the vtable**: the vtable stores the offset to the shared base, which varies depending on the actual class of the object.

Cost: Every access to a virtually-inherited field requires an extra vtable lookup compared to normal field access.

Interface Dispatch (Java)

Interfaces have no fields, only method signatures. Early Java: `invoke_interface` did a linear search through the vtable to find the method—O(n) per call, terribly slow.

Better approach: **Graph coloring on vtable slots**. Methods from the same interface are assigned the same slot positions across all implementing classes. This is feasible because vtables can always be extended with more slots (unlike registers, where k is fixed). Dynamic class loading complicates this (new classes may be loaded at runtime, requiring slot reassignment).

Hot/Cold Field Splitting

Objects with many fields: place frequently accessed ("hot") fields directly in the object, and rarely accessed ("cold") fields behind a pointer. This dramatically improves cache locality because the hot fields are packed tightly together. Accessing cold fields requires an extra pointer dereference plus a likely cache miss. Determine which fields are hot via profiling or JIT statistics.

JIT Compilation (Java)

The Java compiler (`javac`) produces bytecode (a stack-based ISA for the JVM). At runtime, there are several execution strategies:

1. **Interpret**: Walk through bytecode and execute each instruction. Slow but zero startup cost.
2. **Compile everything ahead-of-time (AOT)**: Compile all bytecode to native code before execution. Slow startup (the standard library is huge) but fast execution.
3. **JIT (Just-In-Time)**: Start by interpreting. After observing code being executed multiple times (a "hot" method), compile it to optimized native code. Only compiles what is actually needed. Gathers runtime statistics during interpretation (which branches are taken, which types are used) and optimizes based on actual behavior. Can recompile or deoptimize at any time if assumptions change.

Miscellaneous Exam Topics

procEntryExit Summary

- **procEntryExit1**: View shift (move arguments from calling-convention locations to IR-expected temps/frame slots) and save/restore callee-saved registers.
- **procEntryExit2**: Sink instruction—artificially defines callee-saved regs and return-value regs as live at the function's end so the register allocator does not repurpose them.
- **procEntryExit3**: Emit the actual assembly prologue (function label, SP adjustment) and epilogue (`JR RA` with `SOME([])`).

The Undecidability Pattern

Several compiler problems are undecidable or NP-hard: alias analysis (in general), precise liveness, optimal register allocation. The compiler must use safe approximations:

- **Liveness false positives** (saying live when dead): Causes extra register pressure but code is still correct.
- **Alias analysis false positives** (saying "may alias" when they do not): Misses optimization opportunities but code is still correct.
- **Register allocation failure**: Spill to memory. Slower but correct.

"The Turing cliff is always there"—attempting perfect analysis makes the analyzer run forever.

Control Flow Analysis (k-CFA)

In functional languages, when you see `f(x)`, you may not know what `f` actually is—it could be any lambda that was assigned to `f`. Control flow analysis determines which functions might be called at each call site. Start by conservatively assuming `f` could be any function, then iterate to a fixed point, narrowing down based on the program's structure.

This also applies to object-oriented languages with dynamic dispatch: what method does `obj.method()` actually call? Control flow analysis can narrow the possibilities.

This was Olin Shivers' PhD thesis: k-CFA, where k is the depth of calling context used to distinguish different calls to the same function.

Typing Derivation (Midterm-Style)

Each node in a typing derivation tree corresponds to a typing rule. For a complex expression like `r.x := f(a[3*2]) + g(y.x)`:

The root uses the **assignment rule**: LHS type must equal RHS type, and the result type is unit. The LHS uses the **record field access rule**: `r.x` has type int. The RHS uses the **plus rule**: both operands must be int. The left operand of + uses the **function application rule**: `f` has type int → int, and its argument uses the **array index rule**: `a[3*2]` has type int. The right operand uses function application: `g` has type string → int, and its argument uses record field access: `y.x` has type string.

Show all work for partial credit on the exam.

CPS (Continuation Passing Style)

Confirmed NOT on the final exam. But for reference: in CPS, no function ever "returns." Instead, every function takes an extra argument called a **continuation** (a function representing "what to do next"). Tail calls simply pass the continuation along. Exceptions are implemented as a second continuation (the error handler). CPS is used as an IR in SML/NJ.

Complete Optimization-to-Analysis Reference

Optimization	Driven by Which Analysis
Common Subexpression Elimination (CSE)	Available Expressions
Constant Propagation	Reaching Definitions
Copy Propagation	Reaching Definitions
Dead Code Elimination	Reaching Definitions + Liveness
Live Range Splitting	Reaching Definitions
Register Allocation	Liveness Analysis
Code Hoisting	Very Busy Expressions
LICM (safety check)	Very Busy Expressions + Domination
Induction Variable Elimination	Loop detection via back edges (Domination)